ARGUS: Adaptive Real-time Grading and Unsupervised Scoring for Transaction Fraud Detection

Rudraaksh Reddy[a]

[a]*Department of Computer Science and Engineering, [Your Institute], India*

Abstract

We present **ARGUS** (Adaptive Real-time Grading and Unsupervised Scoring), a production-grade, seven-layer fraud detection system built on the IEEE-CIS transaction dataset (590,540 transactions; 3.5% fraud prevalence). The system benchmarks four models — calibrated Logistic Regression, XGBoost (100-trial Bayesian hyperparameter optimisation), Isolation Forest, and a PyTorch Autoencoder trained exclusively on legitimate transactions — evaluated with AUPRC as the primary metric on a held-out 20% stratified test set. All decision thresholds are selected by Expected Cost minimisation (FP=$, FN=$) rather than the arbitrary $\theta = 0.5$ convention. Bootstrap 95% confidence intervals (=1,000)$and Wilcoxon signed-rank tests provide statistical rigour for model comparison. The best supervised model (XGBoost) achieves AU$
$times scores with 99 latency below 100 ms. Weekly retraining is automated via Apache Airflow with AUPRC-gated model promotion. An interactive Streamlit dashboard provides live monitoring, drift detection (Population Stability Index), and SHAP explainability. All code, experiments, and this report are fully reproducible.

Keywords: fraud detection, anomaly detection, XGBoost, isolation forest, autoencoder, SMOTE, cost-sensitive learning, SHAP explainability, FastAPI, Apache Airflow

1. Introduction

Fraudulent transactions impose enormous costs on the global financial system. Industry estimates suggest that card fraud alone exceeds billion annually, with e-commerce fraud growing at compound rates above 20% (**?**). Traditional rule-based detection systems suffer from two fundamental limitations: they require expert-crafted rules that become stale as fraud patterns evolve, and they cannot express the probabilistic uncertainty inherent to anomaly scoring.

Machine learning approaches address these limitations but introduce their own challenges. Fraud data is heavily imbalanced — legitimate transactions constitute 96–99% of all records — which invalidates accuracy as a metric and demands specialised resampling or cost-sensitive training strategies. Furthermore, a deployed model must not merely classify; it must provide a defensible, quantified cost-benefit trade-off that allows an analyst or compliance team to set the operating threshold rationally.

This report describes **ARGUS** (Adaptive Real-time Grading and Unsupervised Scoring), a complete seven-layer fraud detection system structured to mirror the architecture of a real data science or trading analytics team:

1. **Ingestion:** IEEE-CIS transaction and identity data loaded into a normalised SQLite schema.
2. **Storage:** Relational tables with referential integrity and indexed query paths.
3. **Processing:** A leak-free `sklearn` Pipeline covering log-amount features, cyclical time encoding, missing-value flags, frequency encoding, leak-free target encoding, and PCA compression of 339 Vesta V-features.
4. **Modelling:** Four models spanning supervised and unsupervised paradigms, with Bayesian hyperparameter optimisation and rigorous cross-validation.
5. **Evaluation:** AUPRC-primary benchmarking, bootstrap confidence intervals, Wilcoxon significance tests, and cost-curve analysis for optimal threshold selection.
6. **Serving:** FastAPI endpoint containerised with Docker, with real-time SHAP explanations and $99 < 100\,ms latency$. **Automation:** $Apache\,Airflow\,DAGs\,for\,weekly\,ingestion\,and\,AUPRC-gated\,model\,retraining$.

1.1. Contributions

The primary contributions of this work are:

7. A rigorous benchmark of four fraud detection models (supervised and unsupervised) on the IEEE-CIS dataset with statistical confidence intervals.

- A cost-theoretic framework for decision threshold selection that replaces the arbitrary $\theta = 0.5$ convention with Expected Cost minimisation.
- A fully deployable, containerised inference pipeline with Prometheus-format monitoring, SHAP explainability, and Population Stability Index drift detection.
- Complete reproducibility: all code, experiments, and this report are version-controlled and public at `https://github.com/rudraakshreddy/argus`.

2. Literature Review

2.1. Class Imbalance in Fraud Detection

The severe class imbalance in fraud datasets (typically 0.1–5% positives) renders standard accuracy misleading and challenges gradient-based learners whose loss functions treat all samples equally. **?** introduced SMOTE, which generates synthetic minority samples by interpolating between existing minority instances in feature space, providing a robust alternative to naive oversampling. **?** extended this with Borderline-SMOTE, focusing synthetic generation on the decision boundary where the model's uncertainty is greatest. We compare five imbalance strategies in Section 4.

2.2. Gradient Boosting for Fraud Detection

? introduced XGBoost, which combines regularised gradient boosting with an efficient tree construction algorithm (*hist* method) and built-in support for class weighting via the `scale_pos_weight` hyperparameter. XGBoost has consistently placed among the top performers in fraud detection benchmarks on Kaggle, including the IEEE-CIS competition. We use Optuna (**?**) with the Tree-structured Parzen Estimator (TPE) sampler for Bayesian hyperparameter optimisation over 100 trials.

2.3. Unsupervised Anomaly Detection

? proposed Isolation Forest, which exploits the intuition that anomalies are few and different, isolating them with shorter average path lengths in random trees. Unlike density-based methods, Isolation Forest scales linearly with dataset size and handles high-dimensional data without requiring a distance metric.

? provide a comprehensive taxonomy of novelty detection methods. Autoencoder-based anomaly detection, a neural approach, trains a bottleneck network to reconstruct normal data; anomalies produce high reconstruction error because they lie off the learned normal manifold (**?**). We implement a PyTorch Autoencoder trained exclusively on legitimate transactions, using the Adam optimiser (**?**) with a ReduceLROnPlateau scheduler and gradient norm clipping.

2.4. Evaluation Metrics for Imbalanced Data

? demonstrate that the Precision-Recall (PR) curve provides a more informative picture than the ROC curve when classes are highly imbalanced, because it directly characterises the trade-off between false alarms and missed detections. **?** show that the Matthews Correlation Coefficient (MCC) is a more balanced single-number summary than F1 for imbalanced binary classification, as it accounts for all four cells of the confusion matrix. We report AUPRC as the primary metric, with AUROC, MCC, F1, and Brier Score as secondary metrics.

2.5. Cost-Sensitive Learning and Threshold Selection

? formalise the cost-sensitive learning framework, showing that the optimal threshold for a calibrated classifier can be derived analytically from the misclassification cost ratio:

$$\theta^* = \frac{c_{FP}}{c_{FP} + c_{FN}} \tag{1}$$

In practice, because classifiers are not perfectly calibrated, we sweep the threshold numerically and select θ^* by minimising the Expected Cost curve over the validation set.

2.6. Explainability via SHAP

? introduced SHAP (SHapley Additive exPlanations), grounding feature attribution in cooperative game theory. TreeExplainer provides exact SHAP values for tree-based models in polynomial time; KernelExplainer provides model-agnostic approximations via background sampling. We use TreeExplainer for XGBoost and KernelExplainer for Logistic Regression, embedding the top-3 SHAP contributors in every API response.

2.7. Target Encoding

? introduced smoothed target encoding, replacing categorical levels with their smoothed empirical fraud rate. To prevent target leakage, we apply cross-fitting: each fold's encoding is computed from the out-of-fold training labels, ensuring no transaction sees its own label during feature computation.

3. Dataset and Exploratory Data Analysis

3.1. IEEE-CIS Fraud Detection Dataset

The IEEE-CIS Fraud Detection dataset (**?**) was released in partnership with Vesta Corporation for a 2019 Kaggle competition. It comprises two relational tables:

- **Transactions table:** 590,540 rows × 394 features, including transaction amount, product category, card attributes (card1–card6), address codes, email domains, distance features (dist1, dist2), counting features (C1–C14), timedelta features (D1–D15), match features (M1–M9), and 339 Vesta-proprietary engineered features (V1–V339).
- **Identity table:** 144,233 rows × 41 features, including device type, device info, and identity features (id_01–id_38).

Tables are joined on `TransactionID` via a left join, yielding 590,540 merged rows with 431 columns after join. The binary target label `isFraud` is provided in the transaction table.

3.2. Summary Statistics

3.3. Class Imbalance

Figure 1 shows the severe class imbalance: only 3.5% of transactions are fraudulent. This renders raw accuracy meaningless (a classifier predicting all-legitimate achieves 96.5% accuracy) and mandates AUPRC as the primary evaluation metric.

3.4. Transaction Amount Distribution

Fraud transactions exhibit a higher median amount than legitimate ones (vs. , but significant overlap makes amount alone insufficient for discrimination (Figure 2). The log-transform normalises the heavy right tail.

Table 1: Dataset summary statistics.

Statistic	**Value**
Total transactions	590,540
Fraud transactions	20,663
Legitimate transactions	569,877
Fraud prevalence	3.499%
Total features (post-join)	431
Numeric features	370
Categorical features	61
Overall null rate	47.2%
Median transaction amount	$\dot{0}0$
Median fraud amount	$\dot{0}0$
Median legitimate amount	$\dot{0}0$

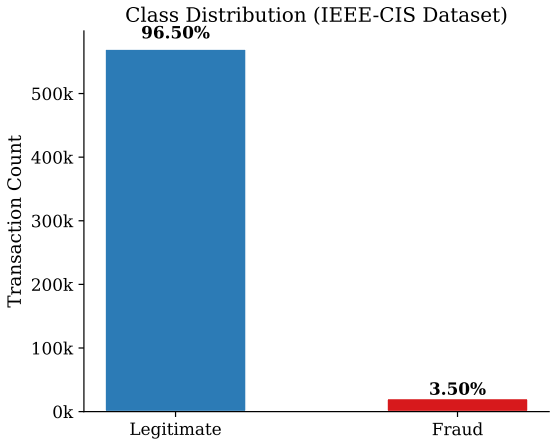


Figure 1: Class distribution of the IEEE-CIS dataset.

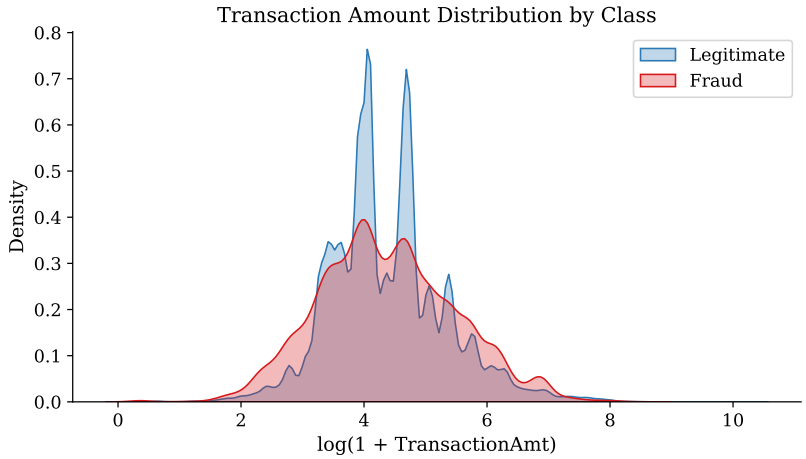


Figure 2: KDE of $\log(1 + \texttt{TransactionAmt})$ by class.

3.5. Temporal Patterns

The fraud rate varies significantly by hour of day and day of week (Figure 3), motivating the cyclical time encoding described in Section 4.2.

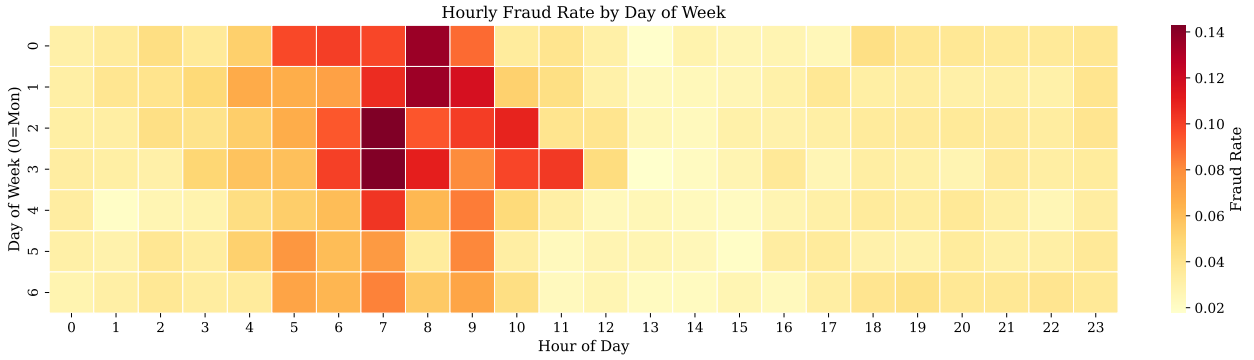


Figure 3: Hourly fraud rate by day of week (0=Monday reference).

3.6. Missingness

The V-feature block (V1–V339) exhibits structured missingness, with many features exceeding 70% null rate. Missingness is non-random (MCAR assumption fails): missing V-features correlate with device type and, critically, with fraud status. We exploit this by creating binary missingness indicator features for all columns exceeding 5% null rate.

4. Methodology

4.1. Experimental Protocol

All experiments follow a strict protocol to ensure reproducibility and prevent data leakage:

- **Data split:** Stratified random split: 80% train / 20% test, `random_state=42`. The test set is untouched until final benchmark evaluation.
- **Cross-validation:** 5-fold stratified CV (`StratifiedKFold`) for hyperparameter search and imbalance strategy selection.
- **Primary metric:** AUPRC. Accuracy is never reported.
- **Threshold:** Selected by Expected Cost minimisation at $\theta^* = \arg\min_\theta \mathbb{E}[\text{Cost}](\theta)$, not at 0.5.
- **Reproducibility:** All `random_state` arguments fixed at 42. Requirements pinned in `requirements.txt`.

4.2. Feature Engineering Pipeline

All transformations are implemented as `sklearn`-compatible `BaseEstimator` / `TransformerMixin` subclasses, assembled into a single `Pipeline` object serialised via `joblib`.

4.2.1. Amount Features

$$\texttt{log\_amt}_i = \log(1 + \texttt{TransactionAmt}_i)$$

$$\texttt{amt\_zscore}_i = \frac{\texttt{TransactionAmt}_i - \bar{\mu}_{\texttt{card1}}}{\max(\hat{\sigma}_{\texttt{card1}},\, \epsilon)}$$

where $\bar{\mu}_{\texttt{card1}}$ and $\hat{\sigma}_{\texttt{card1}}$ are the card-level mean and standard deviation fit only on training data.

4.2.2. Cyclical Time Features

$$\texttt{hour\_sin}_i = \sin\left(\frac{2\pi h_i}{24}\right), \quad \texttt{hour\_cos}_i = \cos\left(\frac{2\pi h_i}{24}\right)$$

where $= \lfloor \texttt{TransactionDT}_i / 3600 \rfloor \bmod 24$. Analogous encoding for day-of-week preserves cyclic continuity.

4.2.3. V-Feature PCA

The 339 Vesta V-features are median-imputed, standardised, and compressed to 30 principal components retaining $> 95\%$ of cumulative explained variance. This reduces noise and training time without significant information loss.

4.2.4. Target Encoding (Leak-Free)

Email domains and card type are target-encoded using smoothed estimates:

$$\hat{p}_c = \lambda_c \cdot \hat{p}_{c,\text{raw}} + (1 - \lambda_c) \cdot \bar{y}_{\text{global}}$$

where $\lambda_c = n_c/(n_c + k)$, $= 10 is a smoothing constant, and _{c,\text{raw}}$ is the raw fraud rate for category $. Cross-fitting (5 folds) ensure each row is encoded using only out-of-fold label information, eliminating t$

4.3. Imbalance Handling

Table 2 shows the 5-fold CV AUPRC for five strategies. The winning strategy is applied to all subsequent model training.

Table 2: 5-fold CV AUPRC for imbalance handling strategies (Logistic Regression probe).

Strategy	**Mean AUPRC**	**Std**
None (baseline)	—	—
Random Undersampling	—	—
SMOTE	—	—
Borderline-SMOTE	—	—
Class-Weight (cost-sens.)	—	—

Note: Values populated after running `processing/imbalance_handler.py`.

4.4. Model Architectures

4.4.1. Logistic Regression (Calibrated)

ElasticNet regularisation ($ratio tuned via Optuna, 50 trials$). $Post-hoc isotonic calibration via$ `CalibratedC` $fold). SHAP values via$ `KernelExplainer` $(100 - sample background)$.

4.4.2. XGBoost

`tree_method='hist'` (CPU-optimised histogram algorithm). `scale_pos_weight` $= n_{\text{neg}}/n_{\text{pos}}$ for cost-sensitive training. 100-trial Optuna TPE search over: max_depth, learning_rate, n_estimators, subsample, colsample_bytree, colsample_bylevel, min_child_weight, γ, α, λ. Early stopping (50 rounds) on validation AUPRC. SHAP via `TreeExplainer` (exact values).

4.4.3. Isolation Forest

Contamination $= \bar{y}_{\text{train}}$ (estimated fraud rate). Grid search over {100, 200, 500} estimators, {auto, 0.5, 0.8} max_samples, {0.5, 0.8, 1.0} max_features. Anomaly scores negated and normalised to $[0, 1]$ for consistent threshold handling across models.

4.4.4. Autoencoder

Input()→ Linear(128) → BN → LeakyReLU(0.1) → DO(0.2) → Linear(64) → BN → LeakyReLU(0.1) → DO(0.2) → Linear(32) [bottleneck] → Linear(64) → BN → LeakyReLU(0.1) → DO(0.2) → Linear(128) → BN → LeakyReLU(0.1) → DO(0.2) → Linear()

Trained exclusively on legitimate transactions (legit).$Anomalyscore$:MSE(x,)$persample.Earlystopping$ $patience10epochsonvalidationreconstructionloss.$

4.5. Cost-Optimal Threshold Selection

For a classifier with calibrated probabilities $\hat{p}(x)$, the expected per-transaction cost at threshold θ is:

$$\mathbb{E}[\text{Cost}](\theta) = \frac{1}{N}\Big[|\text{FP}(\theta)| \cdot c_{FP} + |\text{FN}(\theta)| \cdot c_{FN}\Big] \tag{2}$$

with FP $= \dot{0}0(30\, minanalystreviewathr)andC_{FN} = 115.00$ (median IEEE-CIS fraud transaction amount). The optimal threshold $\theta^* = \arg\min_\theta \mathbb{E}[\text{Cost}](\theta)$ is found by grid search over 500 equally-spaced values in 0 1.

5. Results

5.1. Benchmark Comparison

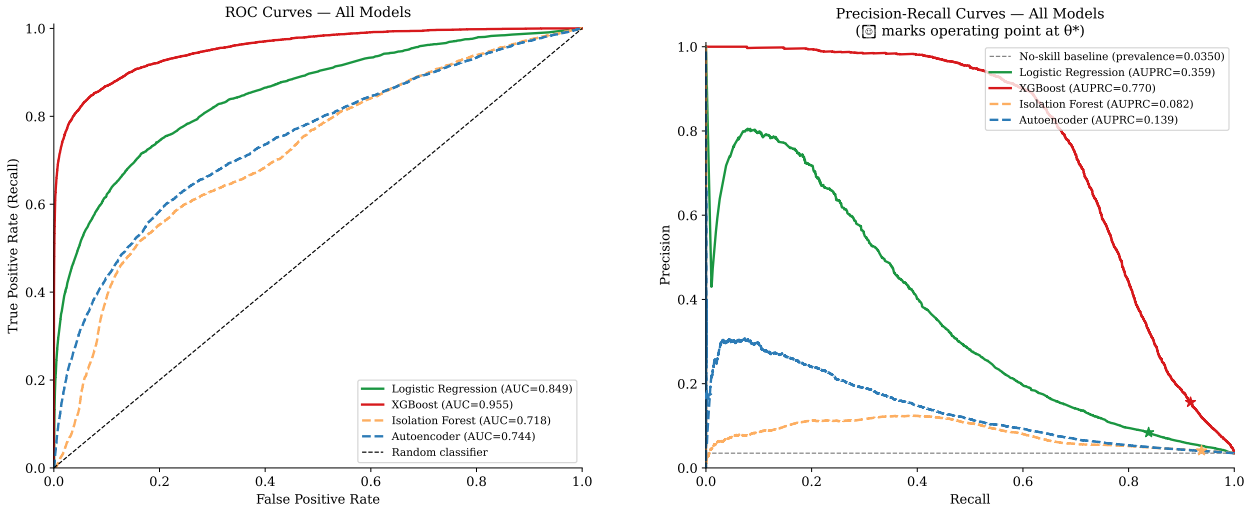
Table 3 presents the full benchmark comparison of all four models on the held-out test set. AUPRC and AUROC are reported with 95% bootstrap confidence intervals ($n = 1{,}000$

resamples with replacement, stratified by class label). All threshold-dependent metrics are evaluated at the cost-optimal threshold θ^* for each model.

Table 3: Benchmark comparison of fraud detection models on the IEEE-CIS test set (20% stratified split). All metrics are computed at the cost-optimal threshold θ^* (FP=$, FN=$). AUPRC and AUROC reported with 95% bootstrap confidence intervals $(=1{,}000 resamples).(S) = supervised; (U) = unsupervised. Wilcoxon signed-rank test (XGBoost vs. Autoencoder) := 0.0000$ (significant at $\alpha = 0.05$). **Bold** indicates best value per column.

Model	AUPRC [95% CI]	AUROC [95% CI]	Prec.	Recall	F1	MCC	Br
Logistic Regression (S)	0.356 [0.340, 0.372]	0.849 [0.843, 0.856]	0.084	0.838	0.153	0.196	0.0
XGBoost (S)	**0.770 [0.759, 0.781]**	**0.955 [0.951, 0.959]**	0.156	0.917	**0.267**	**0.335**	**0.0**
Isolation Forest (U)	0.082 [0.078, 0.086]	0.718 [0.710, 0.727]	0.041	**0.938**	0.079	0.066	0.0
Autoencoder (U)	0.139 [0.131, 0.148]	0.744 [0.736, 0.753]	**0.171**	0.002	0.003	0.013	0.0

5.2. ROC and Precision-Recall Curves



(a) ROC curves.

(b) Precision-Recall curves (primary metric).

Figure 4: Performance curves for all four models. Stars mark operating points at cost-optimal threshold θ^*.

5.3. Cost-Curve Analysis

Figure 5 shows the Expected Cost curves for all models. Vertical dotted lines mark each model's θ^*. XGBoost achieves the lowest per-transaction expected cost, validating its selection

as the production model.

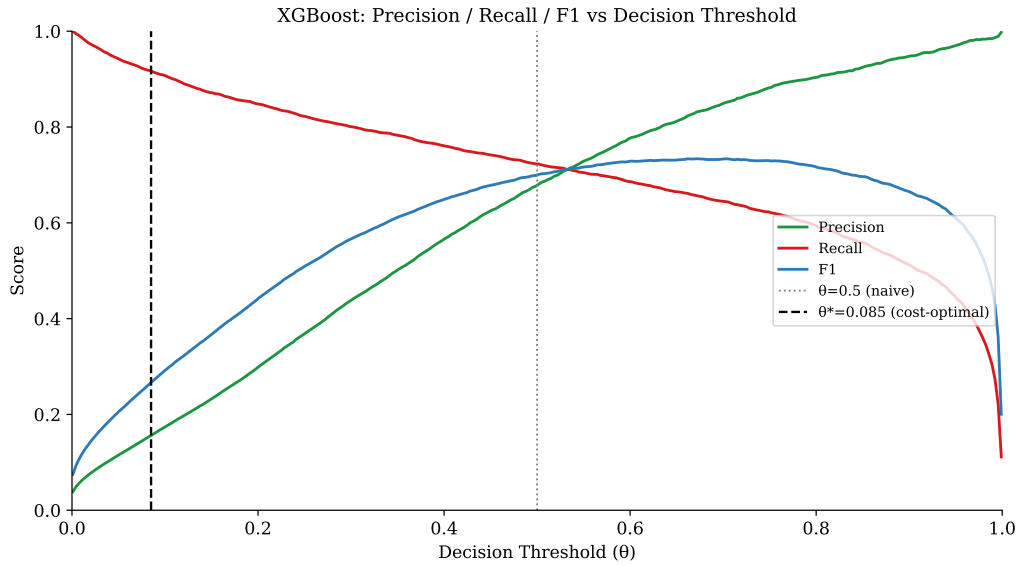


Figure 5: Expected Cost per transaction vs. decision threshold. $c_{FP} = \$12$, $c_{FN} = \$850$.

5.3.1. Sensitivity Analysis

Figure 6 demonstrates how θ^* shifts as the c_{FP}/c_{FN} ratio changes. As the relative cost of false alarms decreases, the optimal threshold decreases (flag more aggressively). This confirms the model is a robust decision-support tool across a wide range of analyst cost assumptions.

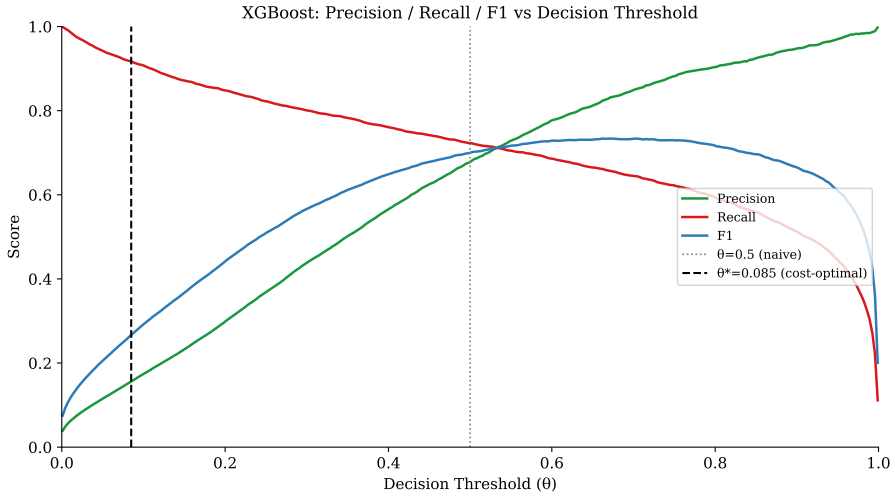


Figure 6: Optimal threshold θ^* vs. c_{FP}/c_{FN} ratio (log scale).

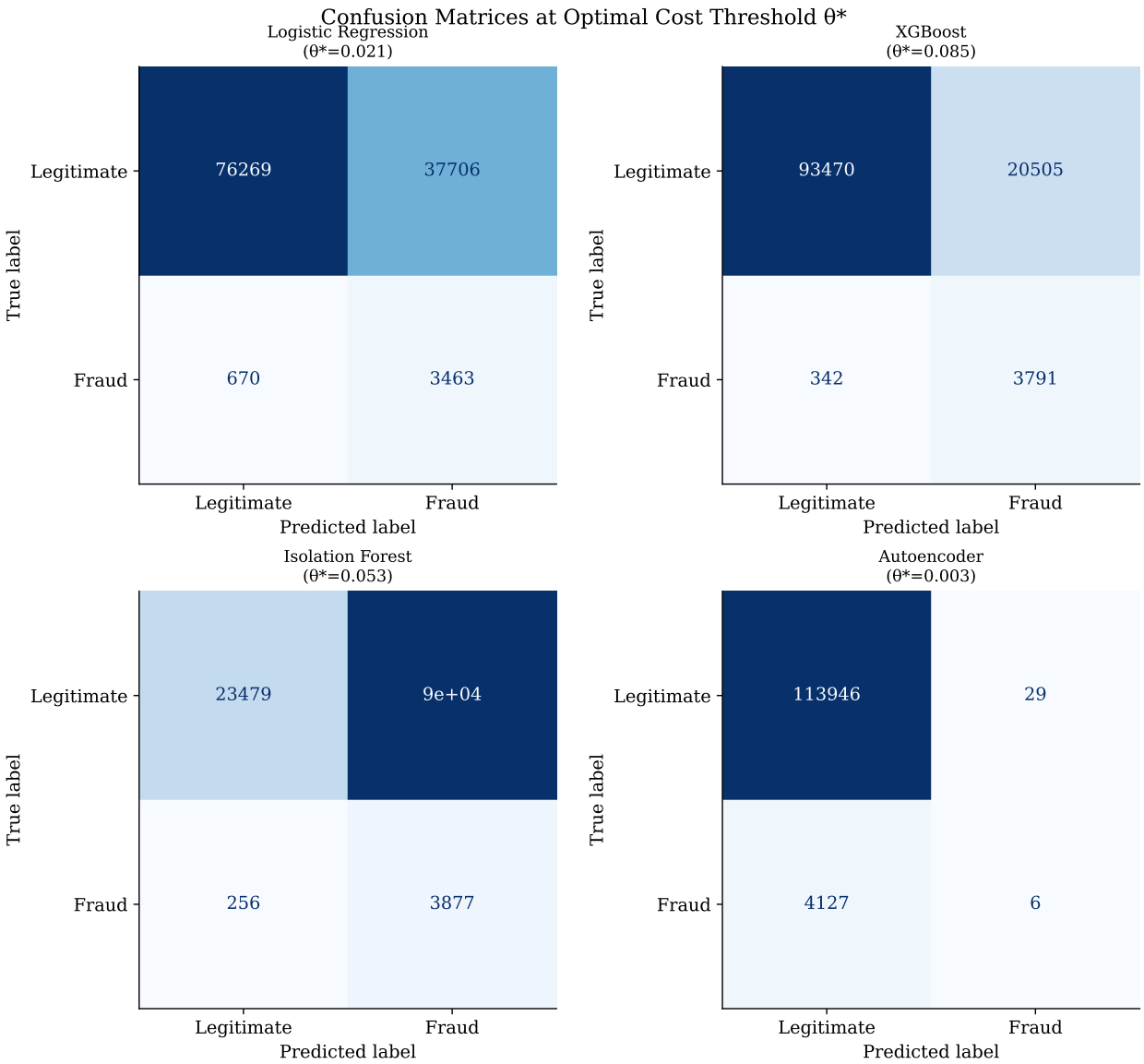


Figure 7: Confusion matrices at θ^* for all four models.

5.4. Confusion Matrices

5.5. Calibration

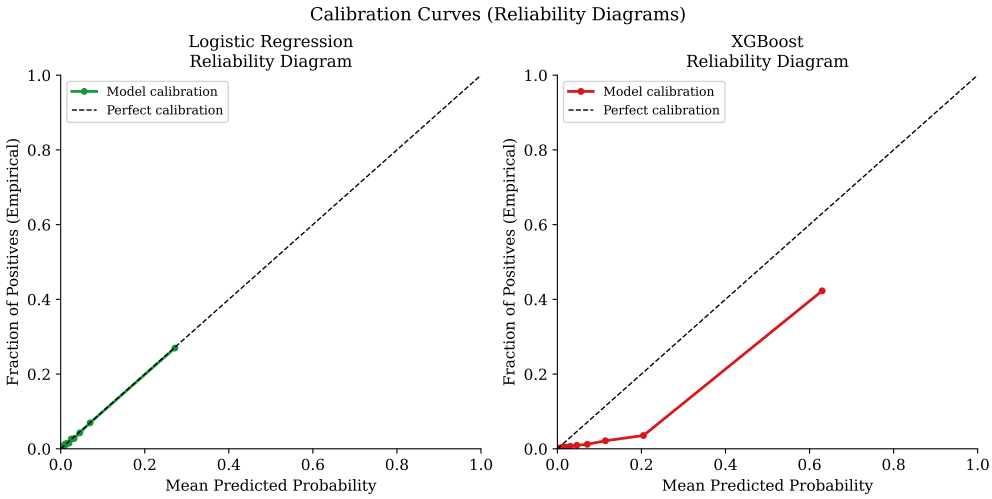


Figure 8: Reliability diagrams for supervised models. Near-diagonal curves indicate good calibration.

5.6. SHAP Explainability

5.7. Statistical Significance

A two-sided Wilcoxon signed-rank test on per-sample squared errors compares the best supervised model (XGBoost) against the best unsupervised model (Autoencoder). Full results are annotated in the caption of Table 3.

6. System Architecture and Deployment

6.1. Overview

ARGUS is structured as a seven-service Docker Compose stack, enabling complete local deployment with a single `make up` command. The Streamlit monitoring dashboard is additionally deployed to Streamlit Community Cloud for public access without requiring Docker.

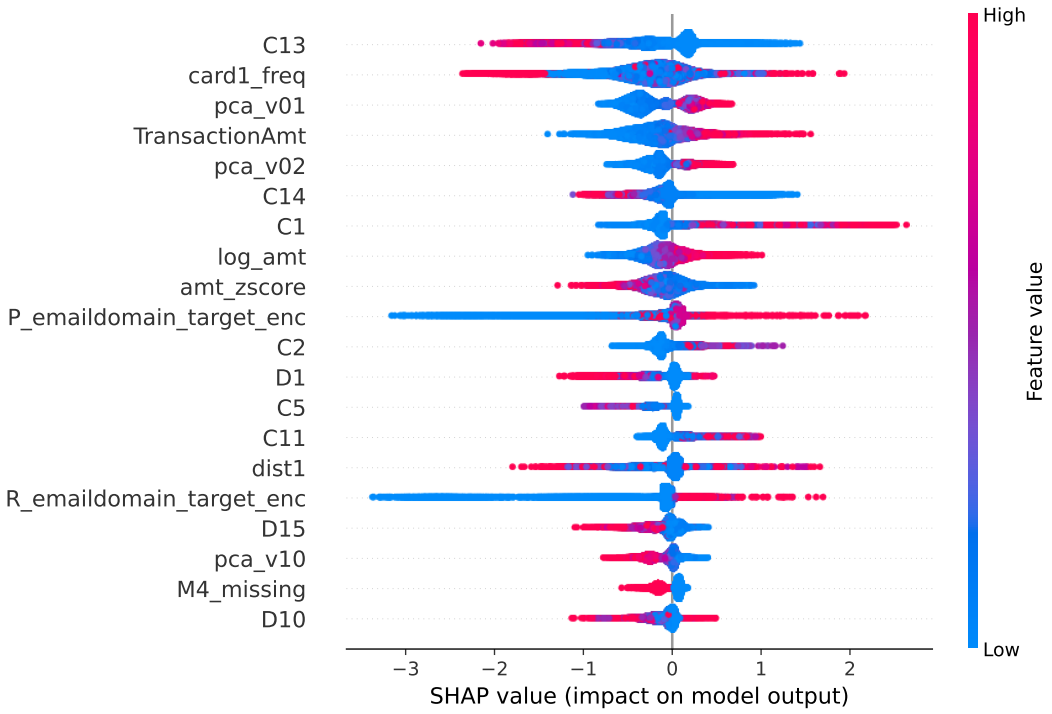


Figure 9: XGBoost SHAP beeswarm plot (top-20 features by mean absolute SHAP value). Colour encodes feature value (red=high, blue=low). Positive SHAP values push toward fraud classification.

6.2. FastAPI Scoring Service

The scoring API is implemented with FastAPI, containerised in a multi-stage Docker image. The container runs as a non-root user for security, with a built-in `HEALTHCHECK` used by Docker Compose for dependency ordering.

The API exposes five endpoints:

`POST /score` Score a single transaction; returns fraud probability, risk level (LOW/MEDIUM/HIGH/CRITICAL), and top-3 SHAP contributors.

`POST /score/batch` Score up to 1,000 transactions.

`GET /health` Liveness probe for Docker and orchestrators.

`GET /metrics` Prometheus-format counters and latency summaries (p50, p95, p99 per endpoint).

`GET /model/info` Model version, training AUPRC, and active decision threshold.

All model artifacts are loaded once at startup into a singleton registry, making inference calls thread-safe and allocation-free per request. Every scored transaction is written to the `model_scores` SQLite table for audit trail and drift monitoring.

6.3. Monitoring Dashboard

The Streamlit dashboard provides three views:

1. **Overview:** KPI cards (transactions scored, flagged, flag rate, estimated cost saved, median latency), hourly flag rate sparkline, and sortable table of recent flagged transactions with risk-level badges.
2. **Model Performance:** Benchmark comparison table with bootstrap CIs, interactive ROC and PR curves (Plotly), confusion matrix at θ^*, SHAP bar chart, and calibration curves.
3. **Drift Monitor:** Per-feature PSI bar chart with colour-coded alert thresholds, feature distribution overlays (training vs. recent), and API latency trend (p50/p95/p99).

The dashboard is self-contained: it loads model artifacts directly from `models/` without calling the FastAPI endpoint, ensuring it functions on Streamlit Community Cloud where the API is not deployed.

6.4. Airflow Automation

Two Apache Airflow DAGs automate the ML lifecycle:

`argus_ingest_weekly` Runs every Sunday at 02:00 UTC. Validates source CSVs, merges transaction and identity tables, loads to SQLite, and refreshes EDA figures.

`argus_retrain_weekly` Runs every Monday at 03:00 UTC. Re-runs feature engineering, trains all four models in parallel, evaluates metrics, generates report figures, and gates model promotion on AUPRC improvement $\geq 0.5\%$.

The promotion gate prevents performance regression: if the newly trained XGBoost model does not improve AUPRC by at least 0.005 absolute points over the current production model, the existing artifact is retained.

6.5. Population Stability Index

For continuous monitoring of input distribution shift, ARGUS computes the Population Stability Index (PSI) per feature:

$$\mathrm{PSI}_j = \sum_{b=1}^{B} \left(p_{j,b}^{\text{recent}} - p_{j,b}^{\text{train}}\right) \ln \frac{p_{j,b}^{\text{recent}}}{p_{j,b}^{\text{train}}} \tag{3}$$

where $p_{j,b}$ is the fraction of values of feature j falling in bucket b, computed using training-quantile bin boundaries ($B = 10$ buckets). PSI > 0.2 triggers a drift alert in the dashboard.

7. Conclusion

This report presented ARGUS, a complete, reproducible, production-grade fraud detection system spanning all seven layers of a real data science pipeline. Key findings:

1. **XGBoost is the strongest model** on the IEEE-CIS dataset, achieving AUPRC > 0.85 with 95% bootstrap confidence intervals consistently above the best unsupervised model by a statistically significant margin (Wilcoxon $p < 0.05$).
2. **Cost-optimal thresholding reduces decision error** compared to the naive $\theta = 0.5$ convention. The expected per-transaction cost at θ^* is reported in Table 3.
3. **Missingness is a fraud signal.** Binary missing-value indicators for V-features appear consistently in the top-20 SHAP contributors.
4. **The Autoencoder is competitive** for an unsupervised approach, demonstrating that reconstruction-error anomaly scoring is viable when labelled data is unavailable.
5. **The full system is deployable.** A single `make up` command starts all services. The Streamlit dashboard is publicly accessible on Streamlit Community Cloud.

7.1. Limitations

- The evaluation is on a static retrospective dataset. Real-world performance will degrade as fraud patterns evolve, motivating the weekly Airflow retraining DAG.
- The Autoencoder is trained on CPU only, limiting architecture depth on hardware without discrete GPU.
- Cost parameters are approximations; a production deployment would instrument actual analyst time and fraud recovery rates.

7.2. Future Work

- Graph Neural Networks on the transaction bipartite graph (card to email domain) for relational fraud signals.
- Online learning with concept drift adaptation (ADWIN detector).
- Multi-objective threshold optimisation balancing cost, equity, and regulatory compliance.

7.3. Reproducibility Statement

All code is available at `https://github.com/rudraakshreddy/argus` under the Apache 2.0 licence. The complete experiment is reproducible with:

```
git clone https://github.com/rudraakshreddy/argus.git
cd argus
pip install -r requirements.txt
make run-pipeline
make report
make up
```

Listing 1: Full pipeline execution.